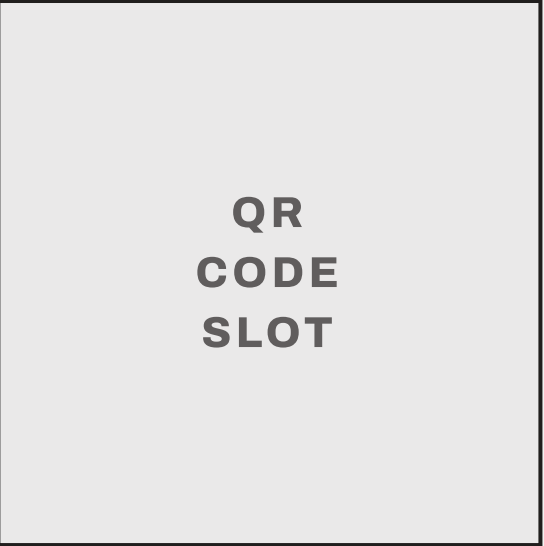


Benchmarking Rust and C Math Libraries for Embedded Robotics

Pascal Harprecht* · Thomas Lübbehüsen* · Wolfgang Hönig — Technical University of Berlin · *equal contribution



PAPER, CODE & FULL TABLES

Across 18 primitives, 6 libraries and 4 build profiles, the language was the weakest predictor in the grid.

1.5×

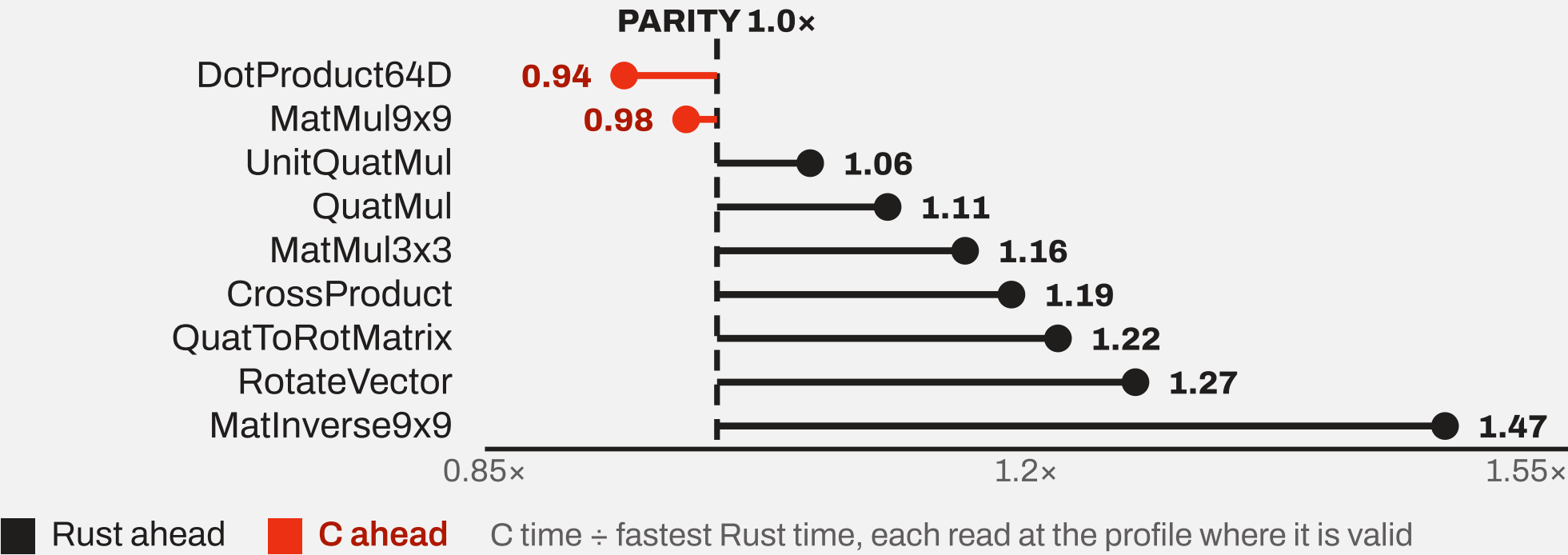
Matched on ABI class and build profile, C and Rust land within 1.5× of each other — in both directions.

OPEN HARDWARE-IN-THE-LOOP BENCH **18 × 909** math primitives × input instances, baked into flash at compile time **6 libraries** glam, nalgebra, micromath, libm vs. CMSIS-DSP and cmath3d **5 chips** STM32F405, RP2040, RP2350, ESP32-S3, nRF52840 — real boards **Cycle-exact** on-chip counters over RTT, no RTOS jitter

01 THE SHAPE OF THE DATA COSTS MORE THAN THE LANGUAGE

STM32F405 · Cortex-M4F, 168 MHz

Account for how the operands are passed and arithmetic is a tie: all nine register- and pointer-passed primitives sit between **0.94×** and **1.47×**.



WHY — THE ABI SORTS EVERY TASK INTO THREE CLASSES

	S0	S1	S2	S3	
vec 3 floats	x	y	z		in registers
quat 4 floats	w	x	y	z	in registers
mat33 9 floats	nine members — five too many				to the stack
9×9 buf 81 floats	one address — the data never moves				by pointer

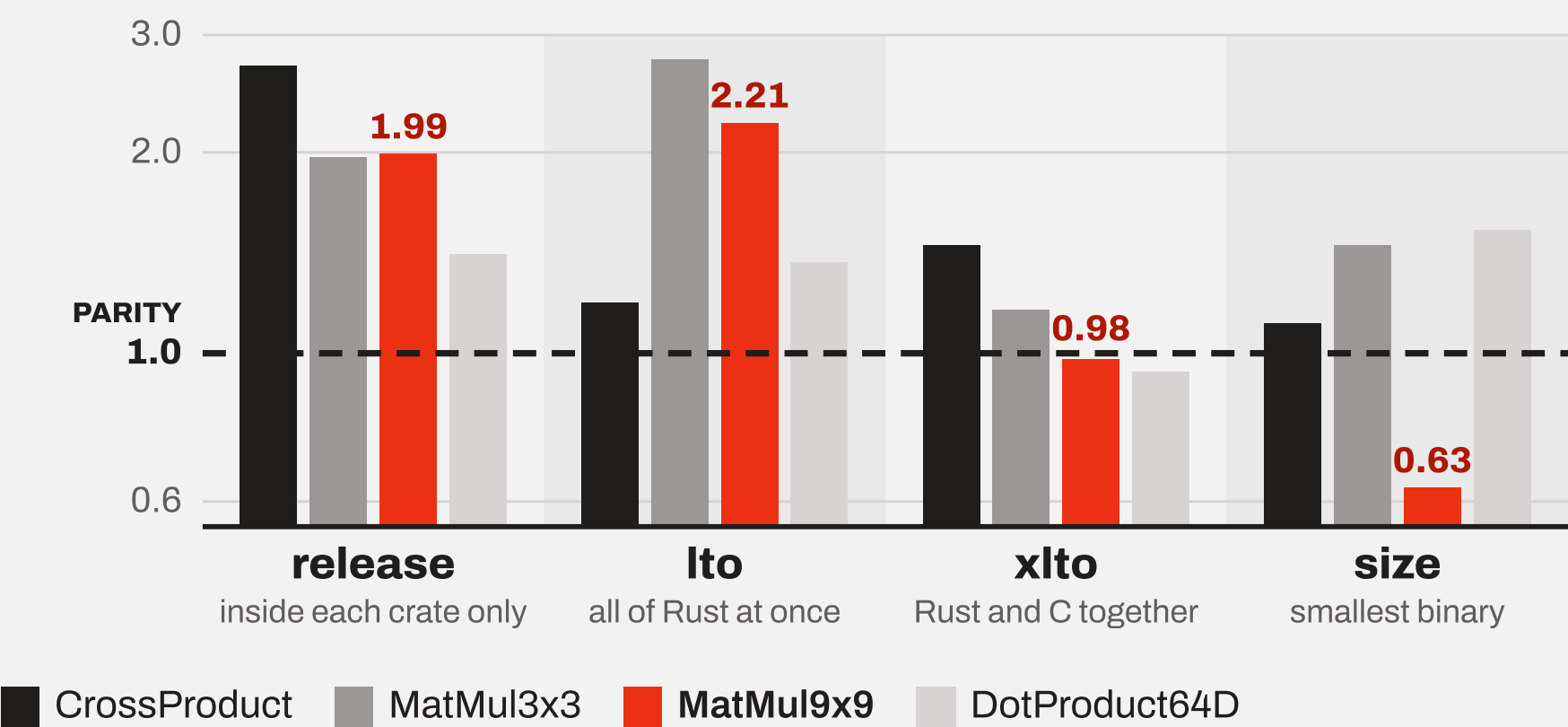
ARM hands a struct over in floating-point registers on one condition: every member is a float, and there are at most four of them. A 3×3 matrix is copied to the stack going in and returned through a hidden **sret** pointer coming out — traffic Rust never pays. Big operands cost nothing at the boundary, but the optimiser treats them differently again.

These three classes are why the chart on the left cannot be read at one profile. Each class is a language comparison only where both sides get equivalent compiler treatment — **that is 02**.

02 THE BUILD PROFILE PICKS THE WINNER

one representative task per class of operands

9×9 matrix multiplication travels from **2.21×** in Rust's favour to **0.63×** in C's on nothing but the link strategy.



WHY — EACH CLASS OF OPERANDS HAS ONE FAIR PROFILE

All in registers CrossProduct, QuatMul, RotateVector, UnitQuatMul 1.19 AT LTO	A 3×3 matrix by value MatMul3x3, QuatToRotMatrix 1.16 AT XLTO	A pointer to a big buffer MatMul9x9, MatInverse9x9, DotProduct64D 0.98 AT XLTO
---	---	--

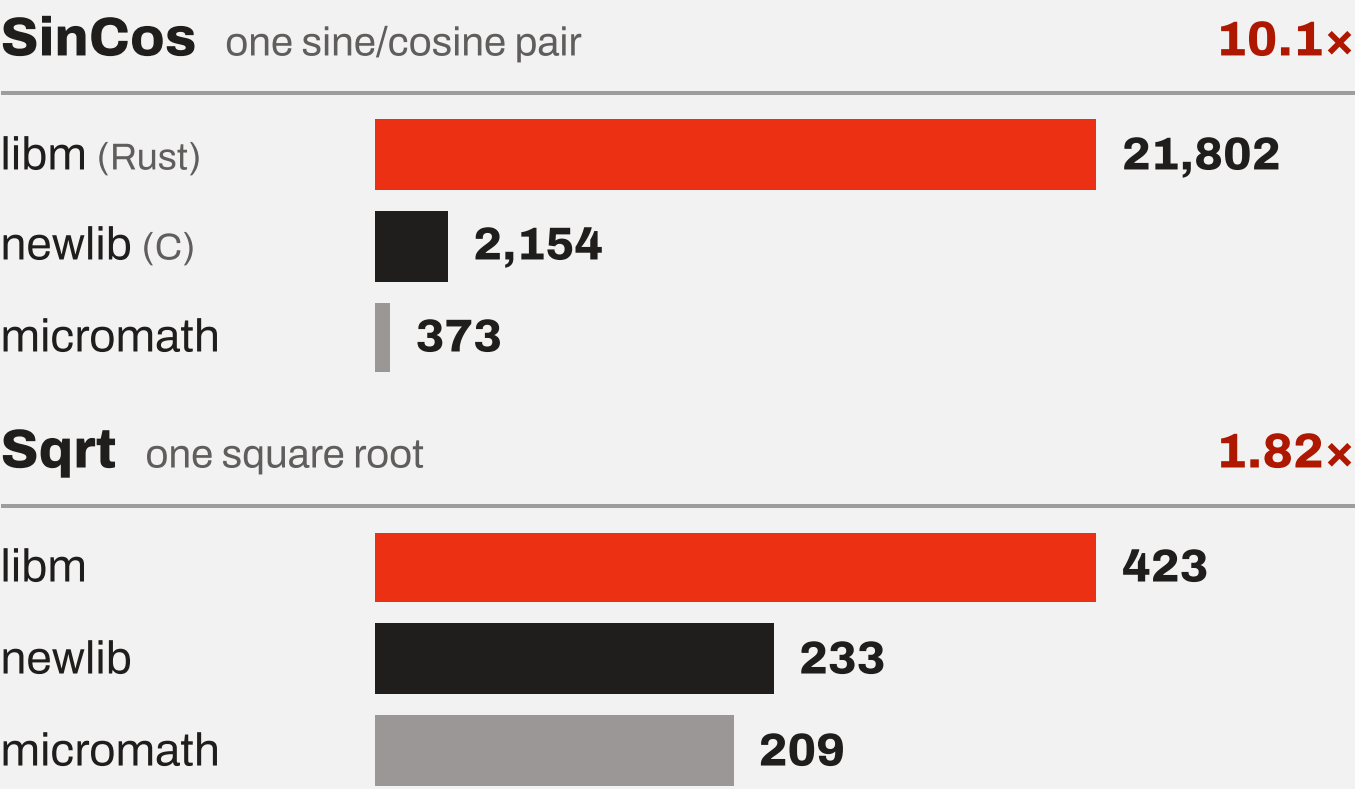
Every other profile is unfair to one side. For register-passed operands, **release** leaves Rust's own scaffolding un-inlined (2.70×) and **xlto** slows C's call site by 26%. For a 3×3 by value, **lto** bills C for AAPCS struct copies (**2.76×**). For pointer operands, **lto** gives Rust fat LTO and the C object nothing (2.21×), and **size** throws away nalgebra's unrolling (0.63×).

lto and **xlto** are this project's own names. Full 4×3 matrix behind the QR.

03 THE MATH BACKEND SWINGS 10×

lto profile · median ns

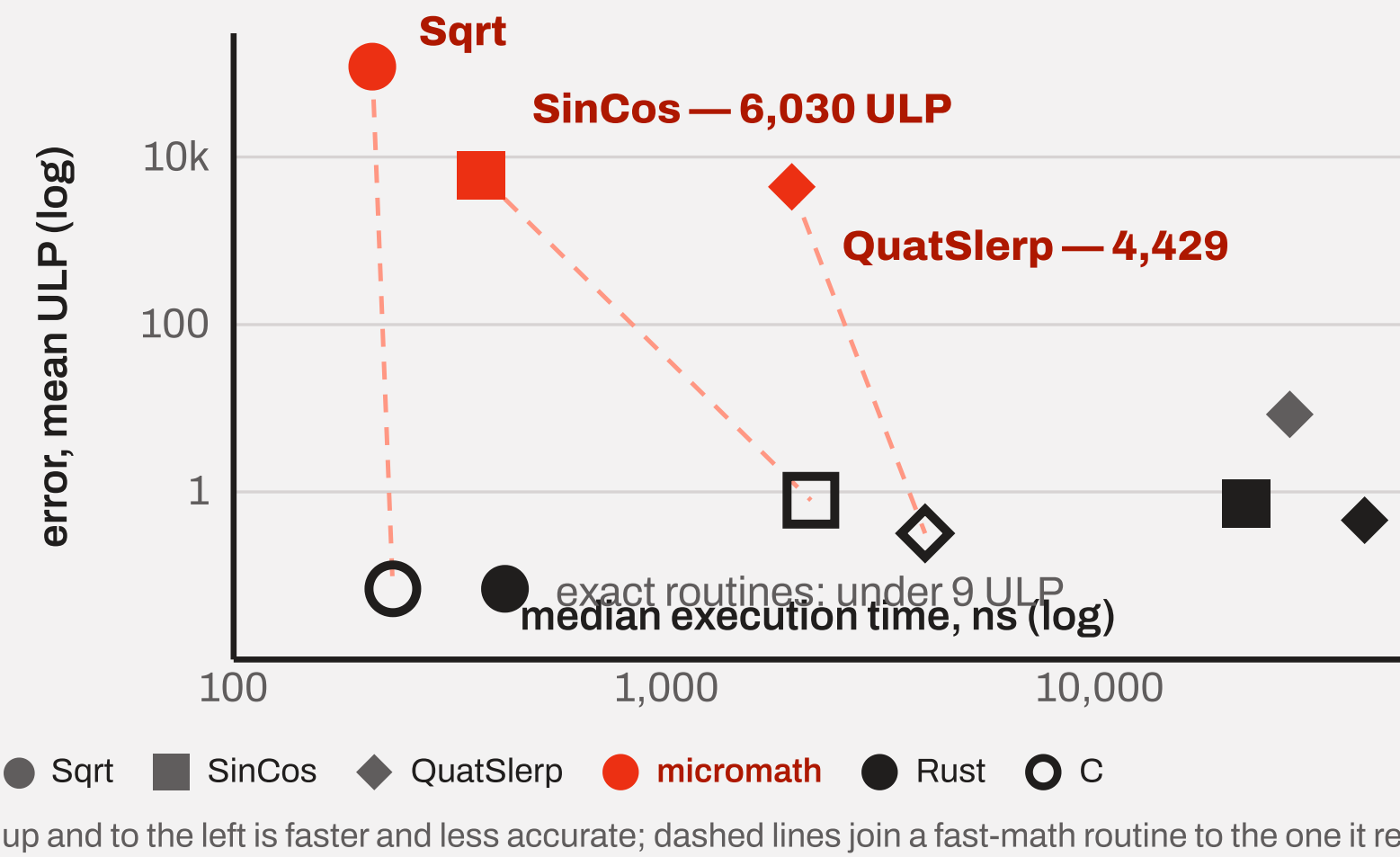
Same operation, three implementations, a **10×** spread — and the gap tracks the library, not the language.



Each group is scaled to its own slowest bar. **newlib** is the ARM GCC toolchain's C math library.

AND THE FAST ROUTE IS PAID FOR IN ACCURACY

120,517 mean ULP error on micromath's Sqrt, against **0.07** for the routine it replaces



WHY — THE FPU IS 32 BITS WIDE

newlib works in **f32**, which the FPU runs in hardware. libm evaluates the same polynomials in **f64**, which it has to emulate in software. That is the whole 10×.

Sqrt is the same story: newlib emits one **vsqrt.f32** instruction; libm has no 32-bit ARM backend at all.

Real code inherits whichever way its function went. **QuatSlerp** calls sine and cosine three times, so cmath3d runs **10.35×** faster than glam.

WHERE IT REVERSES	xlto is not a strict win Of 60 task × library pairs: 20 faster, 26 unchanged, 14 slower . Pure-Rust rows regress too — micromath SinCos +31%.	libm wins three of the five transcendentals Median ns, libm vs. newlib: Exp 598 /1,106 · Ln 566 /1,007 · Atan2 1,091 /1,456. The f64 penalty is not uniform.	Both directions are represented Two of the nine arithmetic primitives land below parity, and the 3×3-by-value class reaches 2.76× at the wrong profile. Nothing here is a clean sweep for either language.
--------------------------	--	---	---

IF YOU ARE CHOOSING A MATH LIBRARY TODAY

- 01 A C-versus-Rust number is tied to the build profile that produced it.
- 02 A slow sqrt or sin/cos in Rust points at the linked math backend, not the language.
- 03 Fast-math crates trade orders of magnitude of accuracy for a few times the speed.

RUNNING NOW Five boards run continuously in CI, including the nRF52840 added	NEXT Closed-loop workloads: an EKF update and a geometric controller step	VERSIONS glam 0.30.10 · nalgebra 0.35.0 · micromath 2.1.0 · libm 0.2.16 · CMSIS-
--	---	--